

Document number	P3349R1
Date	2025-02-10
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Converting contiguous iterators to pointers

- - [Revision History](#)
 - [Introduction](#)
 - [Discussion](#)
 - [Concerns with the proposal](#)
 - [Proposed wording](#)
 - [Acknowledgements](#)
 - [References](#)

Revision History

- R1: Ensure that checked iterators can still detect bounds errors.

Introduction

Iterators that model the `std::contiguous_iterator` concept can be converted to a pointer by calling the `std::to_address` function. However, the standard library is not allowed to do anything useful with such a pointer. We should fix that.

Discussion

C++20 introduced the `std::contiguous_iterator` concept, which can be used to check whether `*(a + n)` is equivalent to `*(std::addressof(*a) + n)`, that is, whether a range denoted by such an iterator contains elements that are stored contiguously in memory.

In theory this is a very useful guarantee, because it allows algorithms to lower a contiguous iterator to a pointer, and operate directly on the underlying memory locations, e.g. by optimizing `std::copy` to `std::memmove`. However, the standard seems to be missing some additional permission to allow such optimizations. Consider a contiguous iterator which throws on increment when it reaches a particular element. This could be used to break out of an algorithm early, e.g.

```
int data[4]{1,2,3,4};
try {
    ranges::for_each(throwing_iterator(data, data+2), data+4,
                    [](int i){ assert(i != 3); });
} catch (...) {
}
```

If the iterator throws when it reaches `data+2` then the assert never fails. This program seems to be valid according to the standard, and must not abort. But this is just silly, and makes contiguous iterators less useful, because the only thing that makes them different from random access iterators is the guarantee of contiguous memory. If that guarantee doesn't allow us to do anything differently, it might as well not exist.

I think the standard library algorithms can conform to the requirements by lowering the iterator for the start of the range to a pointer, then incrementing that iterator until it reaches the sentinel, and if those increments didn't throw, then use `memmove` (or similar) on the raw pointer. But that seems silly too, and may add unnecessary overhead performing those increments just to check if they throw, which will almost certainly never happen in any real program.

Non-standard algorithms *are* allowed to lower contiguous iterators to pointers and operate on the underlying memory directly, because they can just document that that's what they do (or even not document it, but just define it as a feature not a bug). But the algorithms in the C++ standard library are expected to work as specified, and the specification doesn't say that contiguous iterators won't be incremented until they reach the sentinel value.

We should add wording to the standard that says the implementation is allowed to replace any valid contiguous range `r` with something like `[to_address(begin(r)), to_address(begin(r))+size(r))` so that programs cannot rely on any side effects of incrementing or dereferencing the contiguous iterators.

Concerns with the proposal

I initially thought that this optimization would only be allowed for non-empty ranges because I wasn't sure if `to_address(i)` is valid for an empty range `[i, i)` where `i` is a value-initialized contiguous iterator. [LWG 4170](#) resolved that concern. The issue discussion also confirmed that the design intent of contiguous iterators was to permit the optimizations that this proposal aims to legitimize.

On the LWG reflector, Peter Dimov observed that the original change proposed in [P3349R0](#) would remove the ability for "safe" contiguous iterators to detect some logic errors. An iterator which throws an exception (or asserts, or calls a violation handler) when incremented out of bounds can avoid undefined behaviour if an invalid range is passed to e.g. `copy_n`. But if the algorithm immediately lowers the iterator to a pointer, the iterator cannot do its checks. The wording in this R1 revision requires the library to perform `i+n` on the original iterator (not only on a pointer `to_address(i)`), which gives the iterator a chance to check for a bounds overrun.

Proposed wording

The edits are shown relative to [N4988](#).

Modify 25.3.1 [iterator.requirements.general] as indicated:

-8- Most of the library's algorithmic templates that operate on data structures have interfaces that use ranges. A *range* is an iterator and a *sentinel* that designate the beginning and end of the computation, or an iterator and a count that designate the beginning and the number of elements to which the computation is to be applied.

-9- An iterator and a sentinel denoting a range are comparable. A range `[i, s)` is empty if `i == s`; otherwise, `[i, s)` refers to the elements in the data structure starting with the element pointed to by `i` and up to but not including the element, if any, pointed to by the first iterator `j` such that `j == s`.

-10- A sentinel `s` is called *reachable* from an iterator `i` if and only if there is a finite sequence of applications of the expression `++i` that makes `i == s`. If `s` is reachable from `i`, `[i, s)` denotes a valid range.

-11- A counted range `i + [0, n)` is empty if `n == 0`; otherwise, `i + [0, n)` refers to the `n` elements in the data structure starting with the element pointed to by `i` and up to but not including the element, if any, pointed to by the result of `n` applications of `++i`. A counted range `i + [0, n)` is valid if and only if `n == 0`; or `n` is positive, `i` is dereferenceable, and `++i + [0, --n)` is valid.

-12- The result of the application of library functions to invalid ranges is undefined.

-?- For an iterator, `i`, of a type that models `contiguous_iterator` (`[iterator.concept.contiguous]`), library functions are permitted to replace `[i, s)` with `[to_address(i), to_address(i + ranges::distance(i, s))]`, and to replace `i + [0, n)` with `[to_address(i), to_address(i + n))]`.

[*Note* ?]: This means a program cannot rely on any side effects of dereferencing a contiguous iterator `i`, because library functions might operate on pointers obtained by `to_address(i)` instead of operating on `i`. Similarly, a program cannot rely on any side effects of individual increments on a contiguous iterator `i`, because library functions might advance `i` only once. — *end note*]

Acknowledgements

Thanks to Casey Carter, Peter Dimov, Tomasz Kamiński, and Tim Song.

References

[N4988](#), Working Draft - Programming Language -- C++, Thomas Köppe, 2024.